

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No: 8

Roll no: 49 PRN No: 12414235 Name of Student: Saloni Tiwari

Subject Name: Deep Learning

Problem Statement:

Implement Pretrained BERT Model for sentiment analysis or text classification on a sample dataset.

Algorithm:

- **Import Libraries:** Import PyTorch, NumPy, Pandas, Hugging Face Transformers, Datasets, and Scikit-learn for dataset handling, model training, and evaluation.
- **Configure Device:** Set the random seed and check GPU availability for faster model training.
- **Load Dataset:** Load the IMDB movie review dataset containing positive and negative sentiment labels. Select a smaller sample for experimentation.
- **Analyze Dataset:** Display the number of training and testing samples and check the distribution of sentiment classes.
- **Load Pre-trained Model:** Load the pre-trained BERT tokenizer and bert-base-uncased model using Hugging Face Transformers.
- **Tokenize Text:** Convert movie reviews into BERT-compatible token IDs, truncate long sequences, and apply dynamic padding.
- **Configure Classification:** Load BERT with a classification head containing two output neurons for negative and positive sentiment.
- **Define Evaluation Metrics:** Calculate accuracy, precision, recall, and F1-score to evaluate model performance.
- **Train the Model:** Fine-tune BERT on the IMDB training dataset using the AdamW optimizer and cross-entropy loss through the Hugging Face Trainer.
- **Experiment with Epochs:** Train separate models for one and two epochs and compare their evaluation metrics.
- **Evaluate the Model:** Generate predictions on the test dataset and display a classification report.
- **Test Custom Sentences:** Predict sentiment and confidence scores for new movie reviews created for experimentation.
- **Save the Model:** Save the fine-tuned BERT model and tokenizer for future use.

Code:

```
# Assignment 8: Pretrained BERT for Sentiment Analysis
# Import libraries and install required packages
!pip -q install -U transformers datasets accelerate scikit-learn

import random
import numpy as np
import pandas as pd
import torch

from datasets import load_dataset
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    DataCollatorWithPadding,
    Trainer,
    TrainingArguments
)
from sklearn.metrics import (
    accuracy_score,
    precision_recall_fscore_support,
    classification_report
)

# Configure random seed and check computing device
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
print("Device:", "cuda" if torch.cuda.is_available() else "cpu")

# Load the IMDB dataset and select a smaller sample
dataset = load_dataset("stanfordnlp/imdb")
TRAIN_SIZE = 2000
TEST_SIZE = 500
train_ds = dataset["train"].shuffle(
    seed=SEED
).select(range(TRAIN_SIZE))
test_ds = dataset["test"].shuffle(
    seed=SEED
).select(range(TEST_SIZE))
print("Training samples:", len(train_ds))
print("Testing samples:", len(test_ds))

# Analyze sentiment class distribution
print("\nClass Distribution:")
print(
    pd.Series(
        train_ds["label"]
    ).value_counts().sort_index()
)
print("\nExample Review:")
print(train_ds[0]["text"][:300])
print("Label:", train_ds[0]["label"])

# Load the pretrained BERT tokenizer
MODEL_NAME = "bert-base-uncased"
MAX_LENGTH = 256
tokenizer = AutoTokenizer.from_pretrained(
    MODEL_NAME
```

```

)

# Tokenize the movie reviews
def tokenize_function(examples):
    return tokenizer(
        examples["text"],
        truncation=True,
        max_length=MAX_LENGTH
    )
tokenized_train = train_ds.map(
    tokenize_function,
    batched=True
)
tokenized_test = test_ds.map(
    tokenize_function,
    batched=True
)

# Apply dynamic padding to the tokenized inputs
data_collator = DataCollatorWithPadding(
    tokenizer=tokenizer
)

# Configure the sentiment labels
id2label = {
    0: "NEGATIVE",
    1: "POSITIVE"
}
label2id = {
    "NEGATIVE": 0,
    "POSITIVE": 1
}

# Create a fresh pretrained BERT model for each experiment
def create_model():
    return AutoModelForSequenceClassification.from_pretrained(
        MODEL_NAME,
        num_labels=2,
        id2label=id2label,
        label2id=label2id
    )

# Define evaluation metrics
def compute_metrics(eval_pred):
    logits, labels = eval_pred

    predictions = np.argmax(
        logits,
        axis=-1
    )
    precision, recall, f1, _ = precision_recall_fscore_support(
        labels,
        predictions,
        average="binary",
        zero_division=0
    )
    return {
        "accuracy": accuracy_score(
            labels,
            predictions
        ),
        "precision": precision,

```

```

        "recall": recall,
        "f1": f1
    }
}

# Define a reusable training function
def train_model(epochs, output_dir):
    model = create_model()
    training_args = TrainingArguments(
        output_dir=output_dir,
        num_train_epochs=epochs,
        per_device_train_batch_size=8,
        per_device_eval_batch_size=16,
        learning_rate=2e-5,
        weight_decay=0.01,
        eval_strategy="epoch",
        save_strategy="epoch",
        load_best_model_at_end=True,
        metric_for_best_model="f1",
        greater_is_better=True,
        logging_steps=50,
        report_to="none",
        fp16=torch.cuda.is_available(),
        seed=SEED
    )
    trainer = Trainer(
        model=model,
        args=training_args,
        train_dataset=tokenized_train,
        eval_dataset=tokenized_test,
        processing_class=tokenizer,
        data_collator=data_collator,
        compute_metrics=compute_metrics
    )
    trainer.train()
    metrics = trainer.evaluate()
    return trainer, metrics

# Experiment 1: Train BERT for one epoch
trainer_1, metrics_1 = train_model(
    epochs=1,
    output_dir="./bert_1_epoch"
)

# Experiment 2: Train BERT for two epochs
trainer_2, metrics_2 = train_model(
    epochs=2,
    output_dir="./bert_2_epochs"
)

# Compare the evaluation results
comparison = pd.DataFrame({
    "Metric": [
        "Accuracy",
        "Precision",
        "Recall",
        "F1-Score"
    ],
    "1 Epoch": [
        metrics_1["eval_accuracy"],
        metrics_1["eval_precision"],
        metrics_1["eval_recall"],
        metrics_1["eval_f1"]
    ],
}]

```

```

    "2 Epochs": [
        metrics_2["eval_accuracy"],
        metrics_2["eval_precision"],
        metrics_2["eval_recall"],
        metrics_2["eval_f1"]
    ]
})
print("\nExperiment Comparison:")
print(comparison)

# Select the two-epoch model for final evaluation
final_trainer = trainer_2

# Generate predictions and display classification report
pred_output = final_trainer.predict(
    tokenized_test
)
predictions = np.argmax(
    pred_output.predictions,
    axis=-1
)
print("\nClassification Report:")
print(
    classification_report(
        pred_output.label_ids,
        predictions,
        target_names=[
            "NEGATIVE",
            "POSITIVE"
        ],
        digits=4
    )
)

# Define a function to predict sentiment for new sentences
def predict_sentiment(texts):
    final_trainer.model.eval()
    inputs = tokenizer(
        texts,
        return_tensors="pt",
        padding=True,
        truncation=True,
        max_length=MAX_LENGTH
    )
    inputs = {
        k: v.to(final_trainer.model.device)
        for k, v in inputs.items()
    }
    with torch.no_grad():
        outputs = final_trainer.model(
            **inputs
        )
        probabilities = torch.softmax(
            outputs.logits,
            dim=-1
        )
        predictions = torch.argmax(
            probabilities,
            dim=-1
        )
    for text, pred, probs in zip(
        texts,

```

```

        predictions,
        probabilities
    ):
        print("\nText:", text)
        print(
            "Sentiment:",
            id2label[pred.item()]
        )
        print(
            f"Confidence: {probs[pred].item():.2%}"
        )

# Test the model on custom movie reviews
sample_texts = [
    "The movie was amazing and the acting was excellent.",
    "The story was boring and the movie was disappointing.",
    "The visuals were great but the plot was weak."
]
predict_sentiment(sample_texts)

# Save the fine-tuned model and tokenizer
SAVE_PATH = "./bert_sentiment_model"

final_trainer.save_model(
    SAVE_PATH
)
tokenizer.save_pretrained(
    SAVE_PATH
)
print("\nModel saved to:", SAVE_PATH)

```

Output:

1] Loading and sampling the IMDB dataset with class distribution analysis.

```
# Load the IMDB dataset, select a sample, and check the class distribution.

dataset = load_dataset("stanfordnlp/imdb")

TRAIN_SIZE = 2000
TEST_SIZE = 500

train_ds = dataset["train"].shuffle(
    seed=SEED
).select(range(TRAIN_SIZE))

test_ds = dataset["test"].shuffle(
    seed=SEED
).select(range(TEST_SIZE))

print("Training samples:", len(train_ds))
print("Testing samples:", len(test_ds))

print("\nClass Distribution:")
print(pd.Series(train_ds["label"]).value_counts().sort_index())

print("\nExample Review:")
print(train_ds[0]["text"][:300])
print("Label:", train_ds[0]["label"])

*** README.md: 100% ██████████ 7.81k/7.81k [00:00<00:00, 184kB/s]
plain_text/train-00000-of-00001.parquet: reconstructing file: 100% ██████████ 21.0MB / 21.0MB, 2.03MB/s
plain_text/train-00000-of-00001.parquet: downloading bytes: ██████████ 20.9MB, 2.02MB/s
plain_text/test-00000-of-00001.parquet: reconstructing file: 100% ██████████ 20.5MB / 20.5MB, 1.97MB/s
plain_text/test-00000-of-00001.parquet: downloading bytes: ██████████ 20.4MB, 1.96MB/s
plain_text/unsupervised-00000-of-00001.pt(...): reconstructing file: 100% ██████████ 42.0MB / 42.0MB, 4.03MB/s
plain_text/unsupervised-00000-of-00001.pt(...): downloading bytes: ██████████ 41.9MB, 3.98MB/s
Generating train split: 100% ██████████ 25000/25000 [00:00<00:00, 122187.92 examples/s]
Generating test split: 100% ██████████ 25000/25000 [00:00<00:00, 91068.58 examples/s]
Generating unsupervised split: 100% ██████████ 50000/50000 [00:00<00:00, 151836.54 examples/s]
Training samples: 2000
Testing samples: 500

Class Distribution:
0    1800
1    1000
Name: count, dtype: int64
```

2] Tokenizing movie reviews and configuring the pretrained BERT model.

```
# Tokenize the reviews and load the pretrained BERT model for binary classification.

MODEL_NAME = "bert-base-uncased"
MAX_LENGTH = 256

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

def tokenize_function(examples):
    return tokenizer(
        examples["text"],
        truncation=True,
        max_length=MAX_LENGTH
    )

tokenized_train = train_ds.map(tokenize_function, batched=True)
tokenized_test = test_ds.map(tokenize_function, batched=True)

data_collator = DataCollatorWithPadding(tokenizer=tokenizer)

id2label = {0: "NEGATIVE", 1: "POSITIVE"}
label2id = {"NEGATIVE": 0, "POSITIVE": 1}

def create_model():
    return AutoModelForSequenceClassification.from_pretrained(
        MODEL_NAME,
        num_labels=2,
        id2label=id2label,
        label2id=label2id
    )

print("Tokenizer and model configuration ready.")

*** config.json: 100% ██████████ 570/570 [00:00<00:00, 14.4kB/s]
tokenizer_config.json: 100% ██████████ 48.0/48.0 [00:00<00:00, 1.43kB/s]
vocab.txt: 100% ██████████ 232k/232k [00:00<00:00, 893kB/s]
tokenizer.json: 100% ██████████ 466k/466k [00:00<00:00, 3.05MB/s]
Map: 100% ██████████ 2000/2000 [00:03<00:00, 653.44 examples/s]
Map: 100% ██████████ 500/500 [00:00<00:00, 652.25 examples/s]
Tokenizer and model configuration ready.
```

3] Fine-tuning BERT for sentiment classification and displaying training evaluation metrics.

```
model.safetensors: reconstructing file: 100% ██████████ 440MB / 440MB, 38.7MB/s

models.safetensors: downloading bytes: ██████████ 415MB, 33.3MB/s

Loading weights: 100% ██████████ 199/199 [00:00<00:00, 3561.06it/s]

[transformers] BertForSequenceClassification LOAD REPORT from: bert-base-uncased
Key | Status |
-----+-----+
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
cls.seq_relationship.weight | UNEXPECTED |
cls.predictions.bias | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias | UNEXPECTED |
cls.predictions.transform.dense.weight | UNEXPECTED |
cls.seq_relationship.bias | UNEXPECTED |
cls.predictions.transform.dense.bias | UNEXPECTED |
classifier.weight | MISSING |
classifier.bias | MISSING |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task
          [250/250 00:54, Epoch 1/1]

Epoch Training Loss Validation Loss Accuracy Precision Recall F1
-----+-----+-----+-----+-----+-----+
1      0.296404       0.291218   0.872000   0.847328   0.902439   0.874016

Writing model shards: 100% ██████████ 1/1 [00:03<00:00, 3.28s/it]

[transformers] There were missing keys in the checkpoint model loaded: ['bert.embeddings.LayerNorm.weight', 'bert.embeddings.L
[transformers] There were unexpected keys in the checkpoint model loaded: ['bert.embeddings.LayerNorm.beta', 'bert.embeddings.L
          [32/32 00:01]

Training Loss Validation Loss Epoch Accuracy Precision Recall F1
-----+-----+-----+-----+-----+-----+
0.296404       0.291218         1   0.872000   0.847328   0.902439   0.874016

Loading weights: 100% ██████████ 199/199 [00:00<00:00, 4351.12it/s]

[transformers] BertForSequenceClassification LOAD REPORT from: bert-base-uncased
Key | Status |
-----+-----+
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
cls.seq_relationship.weight | UNEXPECTED |
cls.predictions.bias | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias | UNEXPECTED |
cls.predictions.transform.dense.weight | UNEXPECTED |
cls.seq_relationship.bias | UNEXPECTED |
cls.predictions.transform.dense.bias | UNEXPECTED |
classifier.weight | MISSING |
classifier.bias | MISSING |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
          [500/500 01:30, Epoch 2/2]

Epoch Training Loss Validation Loss Accuracy Precision Recall F1
-----+-----+-----+-----+-----+-----+
1      0.320504       0.304636   0.870000   0.822064   0.939024   0.876660
2      0.205970       0.376323   0.888000   0.892562   0.878049   0.885246

Writing model shards: 100% ██████████ 1/1 [00:05<00:00, 5.65s/it]
Writing model shards: 100% ██████████ 1/1 [00:07<00:00, 7.09s/it]

[transformers] There were missing keys in the checkpoint model loaded: ['bert.embeddings.LayerNorm.weight', 'bert.embeddings.LayerNorm.bias',
[transformers] There were unexpected keys in the checkpoint model loaded: ['bert.embeddings.LayerNorm.beta', 'bert.embeddings.LayerNorm.gamma']
          [32/32 00:01]

Training Loss Validation Loss Epoch Accuracy Precision Recall F1
-----+-----+-----+-----+-----+-----+
0.205970       0.376323         2   0.888000   0.892562   0.878049   0.885246

Metric | 1 Epoch | 2 Epochs |
-----+-----+-----+
Accuracy | 0.872000 | 0.888000 |
Precision | 0.847328 | 0.892562 |
Recall | 0.902439 | 0.878049 |
F1-Score | 0.874016 | 0.885246 |
```

4] Evaluating the trained BERT model and predicting sentiment for custom movie reviews.

***	precision	recall	f1-score	support
NEGATIVE	0.8837	0.8976	0.8906	254
POSITIVE	0.8926	0.8780	0.8852	246
accuracy			0.8880	500
macro avg	0.8881	0.8878	0.8879	500
weighted avg	0.8881	0.8880	0.8880	500
Text: The movie was amazing and the acting was excellent.				
Sentiment: POSITIVE				
Confidence: 98.59%				
Text: The story was boring and the movie was disappointing.				
Sentiment: NEGATIVE				
Confidence: 98.64%				
Text: The visuals were great but the plot was weak.				
Sentiment: NEGATIVE				
Confidence: 98.22%				